

PRIVATEFIN

A Privacy-First Explainable Financial Advisory System

Preliminary Report

CM3020 Artificial Intelligence
Project Idea 2: Financial Advisor Bot

University of London | Department of Computer Science
Final Year Undergraduate Project

Chapter 1: Introduction

1.1 Project Concept

PrivateFin is a locally-executed, explainable AI financial advisory application designed to serve general and novice retail investors. The system generates structured, metric-anchored investment recommendations using Meta's Llama 3 large language model, served entirely on-device via Ollama on Apple M1 hardware. Its defining architectural property is Zero Data Egress: at no point during operation is any personal financial query data transmitted to external servers. Every recommendation is accompanied by a mandatory Structured Chain-of-Thought (CoT) reasoning trace, following the Observe → Interpret → Infer → Recommend scaffold, ensuring that all outputs are auditable, explainable, and compliant with the EU AI Act (2024) and GDPR Articles 5, 22, and 25.

This project is developed under Template: CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot, as specified in the University of London project guidelines.

1.2 Motivation

The retail financial technology landscape presents a compounding set of problems for everyday investors. First, high-performance AI advisory systems — most notably BloombergGPT (Wu et al., 2023) and FinGPT (Yang et al., 2023) — route all inference through cloud APIs, meaning that every query a user submits implicitly discloses which securities they are researching, at what frequency, and in what context. Under GDPR Article 4(1), such query data constitutes personal data where it is reasonably linkable to a natural person. Research by the Financial Conduct Authority (FCA, 2023) found that 68% of retail investors are unaware their interaction data with financial platforms is used for commercial model training, rendering informed consent an unreliable compliance mechanism.

Second, the recommendations produced by existing systems are opaque. Platforms such as eToro, Nutmeg, and robo-advisors like Betterment and Wealthify employ black-box portfolio optimisation models that produce Buy / Hold / Sell signals without any derivable audit trail. The EU AI Act (2024), which classifies AI systems used for investment advisory as High-Risk AI Systems under Article 6 and Annex III, mandates under Article 13 that such systems produce outputs sufficiently transparent for users to interpret them appropriately. A system that states only "generated by a neural network" does not satisfy this standard.

Third, access to high-quality financial AI is gatekept by cost. Bloomberg Terminal subscriptions exceed £20,000 per annum, structurally excluding the retail investor segment these tools nominally serve. No major open-source local fintech advisory tool with full explainability currently exists.

PrivateFin is designed to close this Transparency-Privacy Gap through four integrated properties: fully local LLM inference eliminating cloud data transmission; deterministic technical indicator computation providing a verifiable audit trail; structured CoT prompting producing regulatory-grade reasoning traces; and a lightweight Streamlit interface deployable on consumer M1 hardware without any cloud subscription.

1.3 Project Template

This project is implemented under CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot. The template calls for the design and implementation of an AI system capable of providing structured financial recommendations from market data inputs. PrivateFin extends this template by integrating a locally-served large language model (Llama 3 via Ollama) as the reasoning layer, replacing the cloud-based inference pipelines typical of comparable systems, and by incorporating an explicit regulatory compliance objective (EU AI Act Article 13; GDPR Articles 5, 22, and 25) as a first-class design constraint rather than an afterthought.

1.4 Report Structure

Chapter 2 provides a literature review of the academic and technical foundations of PrivateFin's design, covering domain-specific LLMs, specialised NLP methodology, quantitative data frameworks, and the regulatory landscape. Chapter 3 details the system architecture, key design decisions, technology stack, work plan, and evaluation strategy. Chapter 4 describes the prototype implementation, presents evaluation results against the core project claims, identifies limitations, and proposes future improvements.

Chapter 2: Literature Review

2.1 Introduction

The application of Large Language Models (LLMs) to retail finance has exposed a Transparency-Privacy Gap: high-capability AI advisory systems are simultaneously opaque (lacking auditable reasoning traces), privacy-invasive (requiring cloud inference), and inaccessible (proprietary deployment). This review surveys the academic and technical foundations of PrivateFin's design across four domains: domain-specific LLM architectures (2.2), specialised NLP methodology (2.3), quantitative data frameworks (2.4), and the regulatory landscape (2.5). Each section identifies the specific gap that PrivateFin addresses; 2.6 synthesises these into a unified research contribution.

2.2 Domain-Specific Large Language Models

2.2.1 BloombergGPT (Wu et al., 2023)

BloombergGPT is a 50-billion parameter, decoder-only causal language model trained on FinPile — a 363-billion token hybrid corpus comprising 51% proprietary Bloomberg data (earnings transcripts, analyst reports, regulatory filings) and 49% general internet corpora. Its distinguishing contribution is domain-weighted corpus construction rather than architectural novelty. On five domain-specific benchmarks — including Financial PhraseBank sentiment, FiQA question answering, and financial named entity recognition — BloombergGPT consistently outperforms GPT-NeoX-20B and OPT-66B despite comparable or smaller parameter counts, validating domain-weighted training as a more efficient approach than raw scale for financial tasks (Wu et al., 2023).

However, BloombergGPT's limitations are structural and directly motivate PrivateFin's design. It is deployed exclusively as a proprietary SaaS product accessible via Bloomberg Terminal subscriptions (exceeding £20,000 per annum), creating a severe access gatekeeping problem for retail investors. More critically, cloud-side inference means every user query transmits financial intent data to external servers. Under GDPR Article 4(1), such query data constitutes personal data where reasonably linkable to a natural person (GDPR, 2016), and Bloomberg's terms do not preclude its use for commercial model improvement. Finally, BloombergGPT produces no structured reasoning output: recommendations are generated auto regressively without enforced logical scaffolding, constituting the black-box behaviour the EU AI Act (2024) designates non-compliant for high-risk AI applications. PrivateFin addresses all three failures: local execution eliminates data transmission; CoT prompting mandates reasoning traces.

2.2.2 FinGPT (Yang et al., 2023)

FinGPT adopts a parameter-efficient fine-tuning paradigm built on Low-Rank Adaptation (LoRA). LoRA decomposes weight update matrices into two low-rank matrices, reducing trainable parameters by over 99% relative to full fine-tuning, making domain adaptation feasible without Bloomberg-scale compute budgets (Hu et al., 2022). The pipeline ingests real-time data from SEC EDGAR, Twitter/X, Reddit, and news APIs, and fine-tunes foundation models (LLaMA, Falcon, ChatGLM) on financial instruction-following objectives. LoRA-fine-tuned LLaMA-7B variants achieve competitive performance against much larger general-

purpose models, validating targeted adaptation over raw scale — a principle directly adopted by PrivateFin.

FinGPT's practical deployment, however, reconstructs the very cloud dependency it claims to avoid: documented implementations route inference through OpenAI GPT-3.5/4 APIs or Hugging Face Inference endpoints. Data sovereignty is therefore not achieved. Furthermore, FinGPT ingests speculative social media discourse (Reddit, Twitter) alongside structured SEC filings without provenance weighting, making its recommendations unanchored to verifiable market data. When a user asks 'why this stock?', FinGPT cannot produce a metric-traceable answer. PrivateFin resolves this by grounding all LLM reasoning in outputs from a deterministic quantitative layer, ensuring every justification references auditable, reproducible calculations.

2.3 Specialised NLP: FinBERT (Araci, 2019)

FinBERT applies domain-adaptive pre-training to BERT (Devlin et al., 2019), an encoder-only transformer architecture. Unlike the autoregressive decoders above, BERT's bidirectional self-attention conditions each token's representation on all surrounding tokens simultaneously, producing contextualised embeddings well-suited to discriminative classification. Araci (2019) initialises from BERT-base-uncased weights and continues pre-training on a financial corpus comprising Reuters TRC2 articles, earnings transcripts, and analyst reports, adapting internal representations to financial vocabulary (EBITDA, basis points, P/E ratios) and the hedged, forward-looking language that general sentiment lexicons misclassify. A softmax classification head fine-tuned on the Financial PhraseBank (Malo et al., 2014) yields 97.1% accuracy and 96.8% F1 — markedly above VADER and vanilla BERT — demonstrating that domain-specific pre-training yields disproportionate gains even at small model sizes.

FinBERT establishes the foundational principle that domain-targeted training outperforms general-purpose models even with fewer parameters — a principle PrivateFin extends to prompt engineering: saturating the LLM's context with structured financial indicator data produces more accurate qualitative interpretations than vague natural language descriptions. However, FinBERT is architecturally incapable of generative tasks: it outputs a probability distribution over three sentiment classes, which does not constitute an explanation in any regulatory sense. Its typical inference pathway also relies on Hugging Face cloud checkpoints. PrivateFin synthesises FinBERT's domain-specificity insight with the generative capability of a decoder-only LLM, while retaining full local execution.

2.4 Quantitative Data Ingestion and Technical Indicators

2.4.1 yfinance: Privacy-Preserving Market Data

The yfinance library (Aroussi, 2019) retrieves OHLCV (Open, High, Low, Close, Volume) time-series data from Yahoo Finance as Pandas DataFrames. Critically for a privacy-preserving system, yfinance requires no API key registration, eliminating the creation of user-linked data trails on third-party servers — a property absent in commercial alternatives such as Alpha Vantage, Bloomberg API, and Quandl. Its determinism is equally important: given identical parameters, yfinance returns identical data, enabling any historical recommendation to be independently audited. This reproducibility is foundational to PrivateFin's compliance with the EU AI Act's traceability mandate: the quantitative inputs to every LLM prompt can be independently verified from public market data.

2.4.2 Technical Indicator Computation via Pandas

Two indicators, computed deterministically via Pandas (McKinney, 2010), serve as the primary structured inputs to PrivateFin's reasoning layer. The Relative Strength Index (RSI; Wilder, 1978) is defined as $RSI = 100 - [100 / (1 + RS)]$, where RS is the ratio of the average gain to average loss over a rolling 14-period window. RSI values in the range [0, 30] signal oversold conditions; values in [70, 100] signal overbought conditions. The MACD (Appel, 1979) is computed as $EMA(12) - EMA(26)$, with a signal line $EMA(9)$ of the MACD series; a bullish crossover occurs when MACD crosses above its signal line.

Both indicators are closed-form and deterministic: given identical OHLCV input, outputs are always identical. In PrivateFin, these values anchor the CoT prompt, providing objective, verifiable data points that the LLM must reference in its reasoning chain, and constitute the audit trail satisfying the EU AI Act's Article 13 transparency requirement. Chain-of-Thought prompting (Wei et al., 2022) further structures this reasoning: by instructing the model to enumerate relevant indicators, their values, the thresholds evaluated, and the logical inference connecting data to recommendation, PrivateFin produces outputs that are both human-readable and regulatory-compliant.

2.5 Regulatory and Ethical Landscape

2.5.1 EU AI Act (2024): Explainability as Legal Obligation

The EU AI Act (Regulation (EU) 2024/1689) classifies AI systems used for investment advisory as High-Risk AI Systems under Article 6 and Annex III. This triggers obligations under Chapter III: Article 13 requires that such systems be sufficiently transparent for users to interpret outputs appropriately; Article 17 mandates technical documentation enabling reconstruction of the system's reasoning; Article 14 requires human oversight mechanisms. Critically, the Act specifies that explainability must be meaningful — a disclosure stating 'generated by a neural network' is insufficient. The system must articulate the specific features and reasoning steps informing each output. PrivateFin's Structured CoT prompting satisfies this standard directly.

2.5.2 GDPR: Data Sovereignty Architecture

Three GDPR provisions directly shape PrivateFin's architecture. Article 22 grants data subjects the right not to be subject to decisions based solely on automated processing with significant effects — investment recommendations clearly qualify. Articles 5 and 25 establish data minimisation and Privacy by Design respectively: Article 25 mandates that data protection principles be embedded in system design, not applied post hoc. Cloud inference is structurally incompatible with this requirement, as it necessitates transmitting financial query data to external processors. PrivateFin's Ollama-based local inference satisfies Articles 5 and 25 by architectural construction — no personal data leaves the user's device.

2.6 Synthesis and Research Gap

The literature reveals a bifurcated landscape: systems are either high-performance but proprietary and cloud-dependent (BloombergGPT), or open-source but still cloud-reliant at inference time with no deterministic auditability (FinGPT). FinBERT demonstrates the value of domain adaptation but is generatively incapable and similarly cloud-deployed. No existing

system integrates local LLM inference, deterministic quantitative grounding, and structured CoT explainability within a single privacy-preserving architecture.

PrivateFin occupies this gap through four integrated components: (1) quantised Llama 3 via Ollama, enabling LLM-class reasoning on consumer M1 hardware without cloud transmission; (2) a yfinance and Pandas pipeline generating reproducible, auditable technical indicators; (3) Structured CoT prompting producing metric-anchored reasoning traces that satisfy EU AI Act Article 13; and (4) full Privacy by Design compliance under GDPR Articles 5 and 25. The combination constitutes a demonstrably novel contribution: the first retail-facing, fully local, explainable financial advisory system grounded in deterministic market data.

2.7 References

Appel, G. (1979). The Moving Average Convergence-Divergence Method. Signalert Corporation.

Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv:1908.10063.

Aroussi, R. (2019). yfinance. Available at: <https://pypi.org/project/yfinance/>.

Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019, pp. 4171–4186.

EU AI Act (2024). Regulation (EU) 2024/1689. Available at: <https://www.euaiact.com>.

FCA (2023). Financial Promotions Quarterly Data 2023 Q1. Available at: <https://www.fca.org.uk>.

GDPR (2016). Regulation (EU) 2016/679. Available at: <https://gdpr-info.eu>.

Hu, E.J. et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Malo, P. et al. (2014). Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. JASIST, 65(4), pp. 782–796.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of SciPy 2010, pp. 56–61.

Touvron, H. et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.

Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.

Wilder, J.W. (1978). New Concepts in Technical Trading Systems. Trend Research.

Wu, S. et al. (2023). BloombergGPT: A Large Language Model for Finance. arXiv:2303.17564.

Yang, H. et al. (2023). FinGPT: Open-Source Financial Large Language Models. arXiv:2306.06031.

Chapter 3: Design

3.1 Project Overview and Design Objectives

PrivateFin is designed around three core claims, each of which is testable and maps directly to a component of the system architecture. First, the system must preserve user privacy: no personal financial query data may be transmitted to any external server during normal operation. Second, its quantitative recommendations must be grounded in correct, deterministic calculations independently verifiable from public market data. Third, its reasoning must constitute meaningful explainability in the sense required by EU AI Act Article 13, not merely a label attached to an opaque output.

These objectives collectively address the Transparency-Privacy Gap identified in Chapter 2 and position PrivateFin as a novel contribution relative to both proprietary systems (BloombergGPT) and nominally open-source alternatives (FinGPT) that nonetheless reconstruct cloud dependency at inference time.

3.2 System Architecture

PrivateFin implements a four-layer pipeline architecture in which each layer has a single responsibility, is independently testable, and communicates with adjacent layers via well-defined function interfaces. The layers are: Data Ingestion, Indicator Engine, CoT Prompt Builder, and Local LLM Inference. The Streamlit UI orchestrates these layers end-to-end on each user-initiated analysis session.

Layer 1 — Data Ingestion: The `yfinance` library retrieves OHLCV (Open, High, Low, Close, Volume) time-series data for a user-specified ticker symbol and date range. No API key is required, eliminating user-linked data trails on third-party servers. Retrieved data is persisted locally to a CSV file in a `data/` directory, keyed deterministically as `{ticker}_historical.csv`, forming an auditable local cache.

Layer 2 — Indicator Engine: A Pandas engine computes two deterministic technical indicators from the OHLCV close-price series. RSI is computed using a 14-period Exponential Weighted Moving Average (EWMA) formulation (Wilder, 1978), with explicit edge-case handling for pure-uptrend and pure-downtrend conditions. MACD is computed as $\text{EMA}(12) - \text{EMA}(26)$ with a signal line $\text{EMA}(9)$, and classified as bullish, bearish, or neutral relative to the signal line. Both calculations are closed-form and reproducible.

Layer 3 — CoT Prompt Builder: A structured prompt template injects all computed indicator values, their conventional threshold boundaries, user-specified risk profile (conservative, balanced, growth, or speculative), and time horizon (short, medium, or long-term) into a prompt sent to Llama 3. The template enforces the `Observe → Interpret → Infer → Recommend` reasoning scaffold, ensuring the model must articulate specific data-driven steps before arriving at a recommendation.

Layer 4 — Local LLM Inference: Llama 3 is served entirely on-device via Ollama at `localhost:11434`. All inference occurs without network egress beyond the initial read-only `yfinance` OHLCV fetch. If Ollama is unavailable, the system degrades gracefully by displaying a pre-formatted template response, maintaining usability as a data analysis tool in testing and restricted environments.

3.3 Key Technology Decisions

Technology	Role	Justification
Llama 3 (Meta)	Core reasoning LLM	Open-source, quantisable to 4-bit for M1; capable of CoT instruction-following
Ollama	Local model server	Fully local inference with no cloud dependency; simple Python API
yfinance	Market data ingestion	No API key required; deterministic, reproducible OHLCV data for any ticker
Pandas	Indicator computation	Industry-standard; RSI and MACD are closed-form and independently auditable
Streamlit	User interface	Lightweight local UI; no external server; accessible to non-technical users
Python 3.11	Application runtime	Full ecosystem compatibility and native Apple Silicon (M1) support

3.4 Work Plan

The project spans 22 weeks across four topics. Topic 4 (Weeks 1–4) covers system architecture design and evaluation planning. Topic 5 (Weeks 5–13) covers the full build: yfinance data pipeline, Pandas indicator engine, CoT prompt template design, Ollama/Llama 3 integration, Streamlit UI development, and unit and integration testing. A contingency buffer occupies Weeks 12–13 to absorb integration delays. Topic 6 (Weeks 13–18) covers multi-method evaluation: quantitative pipeline correctness, privacy architecture audit, CoT explainability quality assessment, and user study. A second contingency buffer occupies Weeks 17–18. Topic 7 (Weeks 19–22) covers final report writing and submission.

Risk mitigation is built into the plan at three levels. If Llama 3 CoT quality is insufficient, the mitigation is to switch to prompt chaining, decomposing reasoning into multiple sequential prompts. If yfinance is deprecated or returns data gaps, the pipeline is abstracted behind an interface that supports substitution with the Alpha Vantage free tier. If M1 hardware proves a performance bottleneck, quantisation precision can be reduced from 4-bit to 2-bit and the context window shortened.

3.5 Evaluation Plan

The evaluation is designed to test PrivateFin's three core claims through a multi-method approach combining automated quantitative testing, regulatory compliance auditing, and structured user evaluation.

Evaluation Component 1 — Quantitative Pipeline Correctness: RSI and MACD values computed by PrivateFin will be validated against outputs from the TA-Lib reference implementation on 10 historical tickers across three time periods each. Success criterion: 100% agreement to four decimal places. Any deviation indicates a software defect rather than a methodological limitation, since both computations are closed-form.

Evaluation Component 2 — Privacy Architecture Audit: Network traffic will be analysed using Wireshark during a live 10-query evaluation session. All outbound connections will be logged.

Success criterion: zero data transmitted to any endpoint other than Yahoo Finance's public read API.

Evaluation Component 3 — CoT Explainability Quality: A structured rubric-based assessment of 20 generated recommendations across 10 tickers (2 per ticker at different market conditions) will score each CoT output on four dimensions: Factual Accuracy (0–2), Logical Coherence (0–2), Completeness (0–2), and Regulatory Compliance against EU AI Act Article 13 (0–2). Success criterion: mean total score $\geq 6/8$. Outputs scoring below 4/8 will be analysed qualitatively to identify systematic prompt template failures.

Evaluation Component 4 — User Study: A minimum of five participants from the target user groups will query PrivateFin for three tickers each, rate CoT explanation understandability on a five-point Likert scale, and complete the System Usability Scale (SUS; Brooke, 1996). A brief semi-structured interview will capture qualitative feedback on trust and privacy perception. Success criteria: mean SUS score ≥ 70 (above the 'Good' usability threshold) and mean explanation understandability $\geq 4/5$.

3.6 References

Appel, G. (1979). The Moving Average Convergence-Divergence Method. Signalert Corporation.

Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. In Jordan et al. (eds.) Usability Evaluation in Industry. Taylor & Francis.

EU AI Act (2024). Regulation (EU) 2024/1689. Available at: <https://www.euaiact.com>.

FCA (2023). Financial Promotions Quarterly Data 2023 Q1. Available at: <https://www.fca.org.uk>.

GDPR (2016). Regulation (EU) 2016/679. Available at: <https://gdpr-info.eu>.

Wilder, J.W. (1978). New Concepts in Technical Trading Systems. Trend Research.

Chapter 4: Feature Prototype

4.1 Prototype Overview

The prototype implements the complete four-layer pipeline in a working, interactive state. All layers — data ingestion, indicator computation, prompt construction, and local LLM inference — are operational and integrated through a Streamlit user interface. This constitutes a demonstrable end-to-end system rather than a partial scaffold. The most technically significant feature implemented and evaluated in this chapter is the Structured Chain-of-Thought (CoT) reasoning pipeline: the combination of the CoT Prompt Builder and Local LLM Inference layers that transforms deterministic quantitative inputs into an auditable, regulatory-grade reasoning trace. This feature is the primary technical differentiator of PrivateFin relative to existing systems and the most directly testable against the project's core claims.

4.2 Implementation

4.2.1 Data Ingestion and Indicator Engine

The data ingestion module (`data_loader.py`) calls `yf.download(ticker, start, end)`, validates that the returned `DataFrame` is non-empty, flattens any `Multindex` column structure produced by `yfinance` for single-ticker downloads, and persists the result as `{ticker}_historical.csv`. A known `yfinance` edge case is addressed by the following guard: if the `DataFrame`'s column index has more than one level, each column label is extracted from its tuple using list comprehension, producing a flat, single-level index compatible with all downstream operations.

The indicator engine (`indicator_engine.py`) computes `RSI(14)` using an EWMA formulation with smoothing factor $\alpha = 1/14$. Two degenerate conditions are handled explicitly: when average loss is zero (pure uptrend), `RSI` is clamped to 100; when average gain is zero (pure downtrend), `RSI` is clamped to 0; warm-up NaN values are filled with the neutral value of 50.0. `MACD` is computed as `EMA(12) - EMA(26)` with signal line `EMA(9)` and histogram `MACD - Signal`. Trend direction is classified as bullish (`MACD` above signal), bearish (`MACD` below signal), or neutral. `RSI` momentum is classified as oversold (< 30), overbought (> 70), or balanced.

4.2.2 Structured CoT Prompt Construction

The prompt builder (`prompt_builder.py`) constructs a two-part prompt: a module-level constant system prompt and a dynamically assembled user prompt. The system prompt instructs Llama 3 to produce exactly four named sections — Observe, Interpret, Infer, and Recommend — and to base every statement exclusively on the supplied market data. This constant definition ensures identical behaviour across all analysis sessions.

The user prompt is assembled via f-string interpolation, injecting the ticker symbol, risk profile, time horizon, and eight indicator values (close price, session price change, `RSI(14)`, `MACD`, signal, histogram, trend classification, and momentum classification) as structured key-value pairs. This dual injection of both raw numeric values and their human-readable classifications ensures the model has access to both the precise quantitative data and its financial interpretation, reducing the risk of directional misclassification.

4.2.3 Local LLM Inference via Ollama

The Ollama client (`ollama_client.py`) calls a locally-running Llama 3 instance at `localhost:11434` via the Ollama Python library. An `is_available()` check calls `ollama.Client.list()` and returns `False` on any exception, enabling graceful degradation: if Ollama is unavailable, the Streamlit UI displays a warning and renders a pre-formatted template response rather than raising an unhandled exception. This ensures the application remains usable as a data and indicator tool in development and restricted environments.

4.2.4 Streamlit Interface

The Streamlit application (`app.py`) presents four live metric cards (latest close price, RSI(14), MACD, and session price change), an interactive price history chart rendered with Altair, the structured prompt in an expandable pane allowing users to inspect the exact input provided to the model, and the parsed CoT model response divided into its four scaffold sections. Sidebar controls expose ticker symbol, date range, risk profile, time horizon, and Ollama model identifier, making the system configurable without code changes.

4.2.5 Unit Tests

Four unit tests across two modules (`tests/test_indicator_engine.py` and `tests/test_prompt_builder.py`) validate RSI computation correctness, indicator enrichment, trend classification logic, and CoT prompt structural integrity. All tests are executable via `pytest` without any live network or Ollama dependency, enabling continuous integration in offline environments.

4.3 Evaluation

4.3.1 Prototype Success Against Core Claims

Against the three core claims of the project: privacy is preserved (all processing is local; the only external network call is the read-only `yfinance OHLCV` fetch); indicator computation is deterministic and correct (RSI and MACD handle known edge cases and produce stable values); and CoT reasoning is structured and traceable (the `Observe → Interpret → Infer → Recommend` scaffold is enforced by the prompt template and consistently appears in model responses).

Evaluation of the CoT quality used a structured rubric approach based on the four dimensions specified in the Topic 4 design (Factual Accuracy, Logical Coherence, Completeness, and Regulatory Compliance). This rubric-based methodology is well-suited to the AI evaluation domain, where qualitative outputs require structured assessment frameworks analogous to inter-rater reliability approaches used in NLP evaluation (Manning et al., 2008). A sample of three live prototype outputs was assessed across the four rubric dimensions (0–2 per dimension, maximum 8 per output):

Ticker	Factual Accuracy	Logical Coherence	Completeness	Reg. Compliance	Total
AAPL	1/2 (RSI misclassified)	2/2	2/2	2/2	7/8
TSLA	2/2	2/2	2/2	2/2	8/8

Ticker	Factual Accuracy	Logical Coherence	Completeness	Reg. Compliance	Total
MSFT	2/2	1/2	2/2	2/2	7/8

The mean score across the three samples is 7.33/8, exceeding the target threshold of 6/8 specified in the evaluation plan. Completeness scores are uniformly 2/2, confirming that the prompt template reliably enforces the four-stage CoT scaffold. Regulatory Compliance scores are uniformly 2/2, indicating that the outputs satisfy EU AI Act Article 13's requirement for step-by-step reasoning linked to specific data inputs.

4.3.2 Identified Limitations

The primary limitation identified during evaluation is directional misclassification in the Factual Accuracy dimension. In the AAPL live output, the model characterises RSI(14) = 58.75 as 'neutral to slightly oversold'. RSI 58.75 is above the midpoint of 50 and correctly classified as neutral to slightly overbought rather than oversold. This constitutes a model hallucination rather than an indicator computation error: the user prompt provides the correct momentum classification ('balanced') but does not explicitly state the threshold boundaries the model should use to interpret it. Without boundary annotations, Llama 3 appears to default to a vague directional inference rather than a precise threshold-relative classification.

Additional limitations include the absence of data caching (every button press triggers a live yfinance download, introducing unnecessary latency on repeated queries for the same ticker); a restricted indicator set (only RSI and MACD are computed, whereas the proposal identified Bollinger Bands and Stochastic RSI as planned extensions); the absence of a persistent audit trail (each analysis is ephemeral rather than logged to a local SQLite store as required by EU AI Act Article 17); short date range vulnerability (fewer than 26 trading days will produce NaN-only MACD values); and non-deterministic LLM temperature (the Ollama client uses the model's default temperature, which may produce inconsistent outputs on repeated identical inputs).

4.4 Proposed Improvements

The directional misclassification issue can be resolved by extending the user prompt with explicit threshold boundary annotations for each indicator value. For example, replacing the current 'RSI: 58.75, momentum: balanced' injection with 'RSI: 58.75 — neutral zone (between oversold boundary: 30 and overbought boundary: 70)' provides the model with the precise contextual information required to avoid boundary ambiguity. This is a low-cost prompt engineering fix rather than an architectural change.

The caching limitation can be addressed by applying Streamlit's `@st.cache_data` decorator to the `download_stock_data()` function, keyed on ticker and date range. This eliminates redundant network calls on repeated queries while maintaining determinism. LLM temperature should be set explicitly to 0.1 in the Ollama client call to improve output reproducibility across repeated identical prompts, which is important for the formal rubric evaluation planned for the full evaluation phase.

The indicator set should be extended to include Bollinger Bands (20-period, 2-standard deviation) and Stochastic RSI, both of which were identified in the project proposal as planned features. Their inclusion would enrich the model's analytical context and improve the quality

of recommendations in sideways (non-trending) market conditions, where RSI and MACD alone are known to produce false signals (Wilder, 1978). A minimum date range validator should be added to the Streamlit sidebar to prevent users from selecting fewer than 30 trading days, ensuring MACD warm-up requirements are always satisfied.

For the full evaluation phase, the formal TA-Lib validation (RSI and MACD against reference outputs to four decimal places on 10 tickers), the Wireshark privacy audit, and the $n \geq 5$ user study with SUS questionnaire all remain to be conducted. The rubric evaluation should be expanded from the three-sample informal assessment conducted at prototype stage to the full 20-sample formal assessment specified in the evaluation plan, ideally with a second independent scorer to assess inter-rater reliability — a standard practice in NLP evaluation methodology (Manning et al., 2008).

4.5 References

Appel, G. (1979). The Moving Average Convergence-Divergence Method. Signalert Corporation.

EU AI Act (2024). Regulation (EU) 2024/1689. Available at: <https://www.euaiact.com>.

GDPR (2016). Regulation (EU) 2016/679. Available at: <https://gdpr-info.eu>.

Manning, C.D., Raghavan, P. and Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.

Wilder, J.W. (1978). New Concepts in Technical Trading Systems. Trend Research.

Wu, S. et al. (2023). BloombergGPT: A Large Language Model for Finance. arXiv:2303.17564.

Yang, H. et al. (2023). FinGPT: Open-Source Financial Large Language Models. arXiv:2306.06031.